Adaptive Signal Processing and Machine Intelligence Coursework

Prof. Danilo P. Mandic
TA: Sithan Kanna

February 1, 2017

Contents

Guidelines 3

1 Spectrum Estimation 4
- 1.1 Discrete Fourier Transform Basics . . . 5
- 1.2 Properties of Power Spectral Density (PSD) . . . 5
- 1.3 Resolution and Leakage of Periodogram-based Methods . . . 6
- 1.4 Periodogram-based Methods Applied to Real–World Data . . . 6

2 Parametric and Line Spectra 8
- 2.1 Correlation Estimation . . . 8
- 2.2 Spectrum of Autoregressive Processes . . . 10
- 2.3 Principal Component Analysis . . . 11
- 2.4 Real World Signals: Respiratory Sinus Arrhythmia from RR-Intervals . . . 11

3 Adaptive signal processing 13
- 3.1 The Least Mean Square (LMS) Algorithm . . . 13
- 3.2 Adaptive Step Sizes . . . 14
- 3.3 Adaptive Noise Cancellation . . . 15

4 Widely Linear Filtering and Adaptive Spectrum Estimation 17
- 4.1 Complex LMS and Widely Linear Modelling . . . 17
- 4.2 Adaptive AR Model Based Time-Frequency Estimation . . . 19
- 4.3 A Real Time Spectrum Analyser Using Least Mean Square . . . 19

Guidelines

The coursework comprises four assignments, whose individual scores yield 80% of the final mark. The remaining 20% accounts for presentation and organisation. Students are allowed to discuss the coursework but must code their own MATLAB scripts, produce their own figures and tables, and provide their own discussion of the coursework assignments.

General directions and notation:

- The simulations should be coded in MATLAB, a *de facto* standard in the implementation and validation of signal processing algorithms.
- The report should be clear, well-presented, and should include the answers to the assignments in a chronological order and with appropriate labelling. Students are encouraged to submit through Blackboard **(in PDF format only)**, although a hardcopy submission at the undergraduate office will also be accepted.
- The report should document the results and the analysis in the assignments, in the form of figures (plots), tables, and equations, and **not** by listing MATLAB code as a proof of correct implementation.
- The students should use the following notation: boldface lowercase letters (e.g. $\mathbf{x}$) for vectors, lowercase letters with a (time) argument ($x(n)$) for scalar realisations of random variables and for elements of a vector, and uppercase letters (X) for random variables. Column vectors will be assumed unless otherwise stated, that is, $\mathbf{x} \in \mathbf{R}^{N \times 1}$.
- In this Coursework, the typewriter font, e.g. `mean`, is used for MATLAB functions.

Presentation:

- The length limit for the report is 42 pages. This corresponds to ten pages per assignment in addition to one page for front cover and one page for the table of contents, however, there are no page restrictions per assignment but only for the full-report (42 pages).
- The final mark also considers the presentation of the report, this includes: legible and correct figures, tables, and captions, appropriate titles, table of contents, and front cover with student information.
- The figures and code snippets (only if necessary) included in the report must be carefully chosen, for clarity and to meet the page limit.
- Do not insert unnecessary MATLAB code or the statements of the assignment questions in the report.
- For figures, (i) decide which type of plot is the most appropriate for each signal (e.g. solid line, non-connected points, stems), (ii) export figures in a correct format: without grey borders and with legible captions and lines, and (iii) avoid the use of screenshots when providing plots and data, use figures and tables instead.
- Avoid terms like *good estimate, is (very) close, somewhat similar*, etc - use formal language and quantify your statements (e.g. in dB, seconds, samples, etc).
- Note that you should submit two files to Blackboard: the report in PDF format and all the MATLAB code files compressed in a ZIP/RAR format. Name the MATLAB script files according to the part they correspond to (e.g. SEASP_Part_X-Y-Z.m).

Honour code:

Students are strictly required to adhere to the College policies on students responsibilities. The College has zero tolerance to plagiarism. Any suspected plagiarism or cheating (or prohibited collaboration on the coursework, see above) will lead to a formal academic dishonesty investigation. Being found responsible for an academic dishonesty violation results in a discipline file for the student and penalties, ranging from severe reduction in marks to expulsion from College.

1 Spectrum Estimation

Aims: Students will learn to:

- Implement multiple versions of the periodogram and understand their advantages and weaknesses.
- Demonstrate and explain the effects of windowing and sampling on spectral leakage and resolution.
- Interpret the results when applying non-parametric spectrum estimation techniques on real world data.

In non-parametric spectrum estimation, the signal of interest is passed through a bandpass filter with a narrow bandwidth which is swept through the frequency range. The measure of the spectral contents of the input can then be interpreted as the filter output power divided by the filter bandwidth. One typical non-parametric spectrum estimator is the periodogram.

Background. For a discrete time **deterministic** sequence $\{x(n)\}$, with finite energy $\sum_{n=-\infty}^{\infty}|x(n)|^2 < \infty$, the Discrete Time Fourier Transform (DTFT) is defined as

$$X(\omega) = \sum_{n=-\infty}^{\infty} x(n)e^{-\jmath\omega n} \quad \text{(DTFT)}. \tag{1}$$

We often use the symbol $X(\omega)$ to replace the more cumbersome $X(e^{\jmath\omega})$. The corresponding inverse DTFT is given by

$$x(n) = \frac{1}{2\pi}\int_{-\pi}^{\pi} X(\omega)e^{\jmath\omega n}d\omega \qquad \text{(inverse DTFT)}. \tag{2}$$

This can be verified by substituting (2) into (1). The **energy spectral density** is then defined as

$$S(\omega) = |X(\omega)|^2 \qquad \text{(Energy Spectral Density)}. \tag{3}$$

A straightforward calculation gives

$$\frac{1}{2\pi}\int_{-\pi}^{\pi} S(\omega)d\omega = \frac{1}{2\pi}\int_{-\pi}^{\pi}\sum_{n=-\infty}^{\infty}\sum_{m=-\infty}^{\infty} x(n)x(m)e^{-\jmath\omega(n-m)}d\omega = \sum_{n=-\infty}^{\infty}\sum_{m=-\infty}^{\infty} x(n)x(m)\left[\frac{1}{2\pi}\int_{-\pi}^{\pi} e^{-\jmath\omega(n-m)}d\omega\right] = \sum_{n=-\infty}^{\infty}|x(n)|^2. \tag{4}$$

In the process, we have used the equality $\int_{-\infty}^{\infty} e^{\jmath\omega(n-m)}d\omega = \delta_{n,m}$ (the Kronecker delta). Equation (4) can be now restated as

$$\sum_{n=-\infty}^{\infty}|x(n)|^2 = \frac{1}{2\pi}\int_{-\pi}^{\pi} S(\omega) \qquad \text{(Parseval's theorem)}. \tag{5}$$

For **random sequences** we cannot guarantee finite energy for every realisation (and hence no DTFT). However, a random signal usually has a finite average power, and can therefore be characterised by average power spectral density (PSD). We assume zero mean data, $E\{x(n)\} = 0$, so that the autocovariance function (ACF) of a random signal $x(n)$ is defined as

$$r(k) = E\{x(k)x^*(k-m)\} \qquad \text{(Autocovariance function ACF)}. \tag{6}$$

The **Power Spectral Density (PSD)** is defined as the DTFT of the ACF in the following way

$$P(\omega) = \sum_{k=-\infty}^{\infty} r(k)e^{-\jmath\omega k} \qquad \text{Definition 1 of Power Spectral Density}. \tag{7}$$

The inverse DTFT of $P(\omega)$ is given by $r(k) = \frac{1}{2\pi}\int_{-\pi}^{\pi} P(\omega)e^{\jmath k\omega}d\omega$, and it is readily verified that $\frac{1}{2\pi}\int_{-\pi}^{\pi} P(\omega)e^{\jmath k\omega}d\omega = \sum_{l=-\infty}^{\infty} r(l)\left[\frac{1}{2\pi}\int_{-\pi}^{\pi} e^{\jmath(k-l)\omega}d\omega\right] = r(k)$.

Observe that

$$r(0) = \frac{1}{2\pi}\int_{-\pi}^{\pi} P(\omega)d\omega. \tag{8}$$

Since from (6) $r(0) = E\{|x(n)|^2\}$ measures the (average) signal power, the name PSD for $P(\omega)$ is fully justified, as from (8) it represents the distribution of the (average) signal power over frequencies. The second definition of PSD is given by

$$P(\omega) = \lim_{N\to\infty} E\left\{\frac{1}{N}\left|\sum_{n=0}^{N-1} x(n)e^{-\jmath n\omega}\right|^2\right\} \qquad \text{Definition 2 of Power Spectral Density}. \tag{9}$$

1.1 Discrete Fourier Transform Basics

Discrete Fourier Transform (DFT).

As shown in (9), the periodogram can be computed using the DFT coefficients. In this assignment, you will revisit several basic concepts when implementing the DFT in MATLAB and will illustrate the effects of incoherent sampling.

a) Sketch the ideal Fourier (magnitude) spectrum of a 20 Hz sine wave and the theoretical continuous frequency DTFT (magnitude) spectrum for a windowed sine wave at 20 Hz (with an arbitrary window size). [5]

b) Generate a 20 Hz sine wave of length $N = 100$, sampled at 1000 Hz, and plot (using `stem`) the $K = 100$ point and $K = 1000$ point DFT spectra. Comment on whether these plots resemble your sketches in part a). [5]

c) Generate a 24 Hz sine wave of length $N = 100$, sampled at 1000 Hz, and plot (using `stem`) the $K = 100$ point DFT spectrum and explain why a clear peak at 24 Hz is now not present in the spectrum. (*Hint: What is the frequency resolution for a $K = 100$ point DFT?*) Propose a method to solve this, so called, incoherent sampling problem. [5]

1.2 Properties of Power Spectral Density (PSD)

Approximation in the definition of PSD.

Show analytically and through simulations that the definition of PSD in (7) is equivalent to that in (9) under a mild assumption that the covariance sequence $r(k)$ decays rapidly, that is, [5]

$$\lim_{N\to\infty} \frac{1}{N} \sum_{k=-(N-1)}^{N-1} |k||r(k)| = 0. \tag{10}$$

Provide a simulation for the case when this equivalence does not hold. Explain the reasons.

Using DTFT to determine PSD from ACF.

Consider an autocovariance function (ACF) given by

$$r(k) = \begin{cases} \frac{M-|k|}{M}, & |k| \le M-1 \\ 0, & \text{otherwise.} \end{cases} \tag{11}$$

Generate $r(k)$ for $M = 10$ and $M = 128$ in MATLAB, and form a vector of length $L = 256$ as:

$$\mathbf{x} = \left[r(0), r(1), \ldots, r(M-1), 0, \ldots, 0, r(-M+1), \ldots, r(-1)\right]^T \quad \text{(zero padding)}. \tag{12}$$

Verify that the MATLAB command `xf=fft(x)` gives $P(\omega_k)$ for $\omega_k = 2\pi k/L$ (the elements of `xf` should be non-negative and real).

a) Using the zero padding properties of the DTFT, explain whether this particular choice of x is needed for the cases M=10 and M=128. Support your claims by simulation results. [5]

b) The vector `xf` will often contain a small imaginary component due to round-off errors. Demonstrate, through simulations, the improved spectral estimate for the case in part a), when the imaginary part of the spectrum is removed by replacing `xf` by `real(xf)`. [5]

c) The imaginary part should not be removed unless you are sure it is negligible. Show that the command `zf = real(fft(z))` when applied to [5]

$$\mathbf{z} = \left[r(-M+1), \ldots, r(-1), r(0), r(1), \ldots, r(M-1), 0, \ldots, 0\right]^T \tag{13}$$

gives erroneous "spectral" values. Explain why this is the case.

d) The use of the `fft` command gives spectral estimates in the range $[0, 2\pi]$ instead of the more convenient $[-\pi, \pi]$. Use the MATLAB command `fftshift` to exchange the first and second half of the FFT output to correspond to the range $[-\pi, \pi]$. Similarly, the `fftshift` can be used to center the ACF around the zero-lag. Experiment with this command for the exercises above. What frequency vector w is needed to ensure that the command `plot(w,fftshift(fft(x)))` gives the spectral values at correct frequencies? Repeat the exercise for the values of ACF at different time lags for the function `stem(n,fftshift(ifft(xf)))`. Does the result depend on whether the vectors are even or odd in length? [5]

1.3 Resolution and Leakage of Periodogram-based Methods

Consider the signal

$$x(n) = a_1 \sin\left(f_0 2\pi n + \phi_1\right) + a_2 \sin\left((f_0 + \alpha/N)2\pi n + \phi_2\right) + w(n), \qquad w \sim \mathcal{N}(0, \sigma^2). \tag{14}$$

where $w(n)$ is real-valued Gaussian noise with zero mean and variance σ^2, and $f_0 = 0.2$.

a) Find empirically the 3 dB width of the main lobe **of the Bartlett window** $W_B(\omega)$ as a function of N, to confirm that its width is proportional to $1/N$. Also, compute the peaks of the sidelobes (in dB) as a function of N. Verify this through plots of the magnitude of $W_B(\omega)$ for several values of N; use both linear and dB scales in your plots. [10]

b) To eliminate the statistical variation in the periodogram, so that the bias properties can be studied, set $\sigma^2 = 0$. Also, set $a_1 = a_2 = 1$, $\phi_1 = \phi_2 = 0$ and $N = 256$. Plot the (zero-padded) periodogram of $x(n)$ for various α and determine the resolution threshold (i.e. the minimum value of α for which the two frequency components in (14) can be resolved). Comment on this value of α. [5]

c) Repeat b) for a Hamming-windowed periodogram. [5]

We next analyse the effects of *leakage* on the periodogram estimate. Leakage is readily visualised when estimating two sinusoidal components which are well separated in frequency, but have greatly differing amplitudes.

d) Use (14) to generate a sinusoidal sequence for $\alpha = 4$, $\sigma^2 = 0$ and $\phi_1 = \phi_2 = 0$. Set $a_1 = 1$ and vary a_2 (e.g., $a_2 \in \{1, 0.1, 0.01, 0.001\}$). Compute the periodogram using a rectangular window and comment on its ability to identify the second sinusoidal term in the spectral estimate. Does the amplitude threshold for the identification of the second sinusoidal term change substantially for $\alpha = 12$? [5]

e) Explain your results in d) in connection with the amplitude of the Fourier transform of the Bartlett window at frequencies corresponding to α/N for $\alpha = 4$ and $\alpha = 12$. [5]

f) For practical purposes, we may desire to use a window with a constant sidelobe level which can be chosen by the user. The Chebyshev window is one such choice (see the MATLAB command `chebwin`). Repeat d) using the Chebyshev window to show that the two sinusoidal components in (14) can be resolved using a windowed periodogram. Compare this result with the Blackman-Tukey method. [10]

1.4 Periodogram-based Methods Applied to Real–World Data

Now consider two real–world datasets: a) The sunspot time series[1] and b) an electroencephalogram (EEG) experiment.

a) Apply one periodogram-based spectral estimation technique (possibly after some preprocessing) to the sunspot time series. Explain what aspect of the spectral estimate changes when the mean and trend from the data are removed (use the MATLAB commands `mean` and `detrend`). Explain how the perception of the periodicities in the data changes when the data is transformed by first applying the logarithm to each data sample and then subtracting the sample mean from this logarithmic data. [10]

The basis for brain computer interface (BCI).

b) The electroencephalogram (EEG) signal was recorded from an electrode located at the posterior/occipital (POz) region of the head. The subject observed a flashing visual stimulus (flashing at a fixed rate of X Hz, where X is some integer value in the range $[11, \ldots, 20]$). This induced a response in the EEG, known as the steady state visual evoked potential (SSVEP), at the same frequency. Spectral analysis is required to determine the value of 'X'. The recording is contained in the `EEG_Data_Assignment1.mat` file[2] which contains the following elements: [10]

- `POz` – Vector containing the EEG samples (expressed in Volts) obtained from the POz location on the scalp,
- `fs` – Scalar denoting the sampling frequency (1200 Hz in this case).

Read the `readme_Assignment1.txt` file for more information.

Apply the standard periodogram approach to the entire recording, as well as the averaged periodogram with different window lengths (10 s, 5 s, 1 s) to the EEG data. Can you identify the the peaks in the spectrum corresponding to SSVEP? There should be a peak at the same frequency as the frequency of the flashing stimulus (integer X in the range $[11, \ldots, 20]$), known as the fundamental frequency response peak, and at some integer multiples of this

[1] Included in MATLAB, use `load sunspot.dat`

[2] Download the EEG recording from `http://www.commsp.ee.ic.ac.uk/~mandic/EEG_Data.zip`

value, known as the *harmonics* of the response. It is important to note that the subject was tired during the recording which induced a strong response within 8-10 Hz (so called alpha-rhythm), **this is not the SSVEP**. Also note that a power-line interference was induced in the recording apparatus at 50 Hz, **and this too is not the SSVEP**. To enable a fair comparison across all spectral analysis approaches, you should keep the number of frequency bins the same.

Hint: It is recommended to have 5 DFT samples per Hz.

How does the standard periodogram approach compare with the averaged periodogram of window length 10 s?

Hint: Observe how straightforward it is to distinguish the estimated SSVEP peaks from other spurious EEG activity in the surrounding spectrum.

In the case of averaged periodogram, what is the effect of making the window size very small, e.g. 1 s?

2 Parametric and Line Spectra

Aims: Students will learn to:

- Perform practical spectrum estimation using parametric models.
- Understand the benefits and drawbacks of parametric and line spectra.
- Implement and analyse signal with time-varying spectra.

2.1 Correlation Estimation

Unbiased correlation estimation and preservation of non-negative spectra. Recall that the correlation-based definition of the PSD leads to the so-called *correlogram spectral estimator* given by

$$P(\omega) = \sum_{k=-(N-1)}^{N-1} \hat{r}(k) e^{j\omega k} \tag{15}$$

where the estimated autocorrelation function $\hat{r}(k)$ can be computed using the **biased** or **unbiased** estimators given by

$$\textbf{Biased: } \hat{r}(k) = \frac{1}{N} \sum_{n=k+1}^{N} x(n)x^*(n-k) \tag{16}$$

$$\textbf{Unbiased: } \hat{r}(k) = \frac{1}{N-k} \sum_{n=k+1}^{N} x(n)x^*(n-k) \qquad (0 \leq k \leq N-1). \tag{17}$$

Although it may seem that the unbiased estimate is more appropriate as its mean matches the true mean of PSD, observe that this estimate (despite being exact) can be highly erratic for larger lags k (close to N), where fewer samples are available to estimate the PSD. As a consequence, the ACF may not be positive definite, resulting in negative PSD values.

a) Write a MATLAB script which calculates both biased and unbiased ACF estimates of a signal and then use these ACF estimates to compute the corresponding correlogram in Eq. (15). Validate your code for different signals e.g. WGN, noisy sinusoidal signals and filtered WGN. Explain how the spectral estimates based on (16)-(17) differ from one another? In particular, how does the correlogram corresponding to the unbiased ACF estimates behave for large lags (i.e. k close to N)? Does the unbiased ACF estimate result in negative values for the estimated PSD? [10]

Plotting the PSD in dB. Depending on the estimation approach, the spectral estimate $\hat{P}(\omega)$ can be asymptotically unbiased with variance $\mu P^2(\omega)$, where $\mu > 0$ is a constant. When several realisations of a random signal are available, it is possible to present the estimate PSD as a confidence interval defined by $\hat{P}(\omega) \pm \mu\hat{\sigma}_{P(\omega)}$, where $\hat{P}(\omega)$ and $\hat{\sigma}_{P(\omega)}$ are respectively the mean and standard deviation of the estimated PSDs of the available observations. A drawback of this approach is that, as stated earlier, the standard deviation is proportional to the value of the PSD and therefore the confidence interval widens in zones where the PSD increases, and it is these parts that we are particularly interested in. Fig. 1 shows an overlay plot of 100 realisations of the PSD of two sinusoids immersed in i.i.id WGN showing the mean (top), and the standard deviation of the set (bottom).

For ease of presentation, by plotting the PSD estimates in decibels we observe a more condensed realisation due to the contraction property of the logarithm.

b) Use your code from the previous section (only the **biased** ACF estimator) to generate the PSD estimate of several realisations of a random process and plot them as in Fig. 1. Generate different signals composed of sinusoids corrupted by noise and elaborate on how disperse are the different realisation of the spectral estimate. [5]

c) Plot your estimates in dB, together with their associated standard deviation (again as in Fig. 1 for comparison). How much spread out are the estimates now? Comment on the benefits of this representation. [5]

Frequency estimation by MUSIC. In order to accurately estimate the spectrum of closely-spaced sine waves using the periodogram, a large number of samples N is required since the frequency resolution of the periodogram is proportionate to $1/N$. On the other hand, subspace methods assume a harmonic model consisting of a sum of sine waves, possibly complex, in additive noise. In this setting, the noise is also complex-valued.

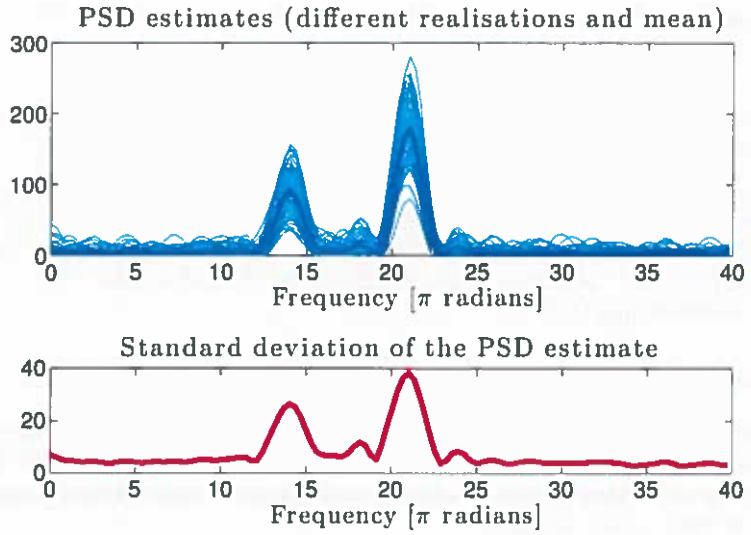


Figure 1: PSD estimates of two sinusoids immersed in noise. Top: An overlay plot of 100 realisations and their mean. Bottom: Standard deviation of the 100 estimates.

For illustration, consider a complex-valued signal of 30 samples in length, generated using the following code:

```
n = 0:30;
noise = 0.2/sqrt(2)*(randn(size(n))+1j*randn(size(n)));
x = exp(1j*2*pi*0.3*n)+exp(1j*2*pi*0.32*n)+ noise;
```

The signal consists of two complex exponentials (sine waves) with frequencies of 0.3 Hz and 0.32 Hz and additive complex white Gaussian noise. The noise has zero mean and variance of 0.2.

The spectral estimate using the periodogram (rectangular window, 128 frequency bins and unit sampling rate) is shown in Fig. 2. Observe that the periodogram was not able to identify the two lines in the spectrum; this is due to the resolution of the periodogram being proportionate to $1/N$, which is greater than the separation between the two frequencies.

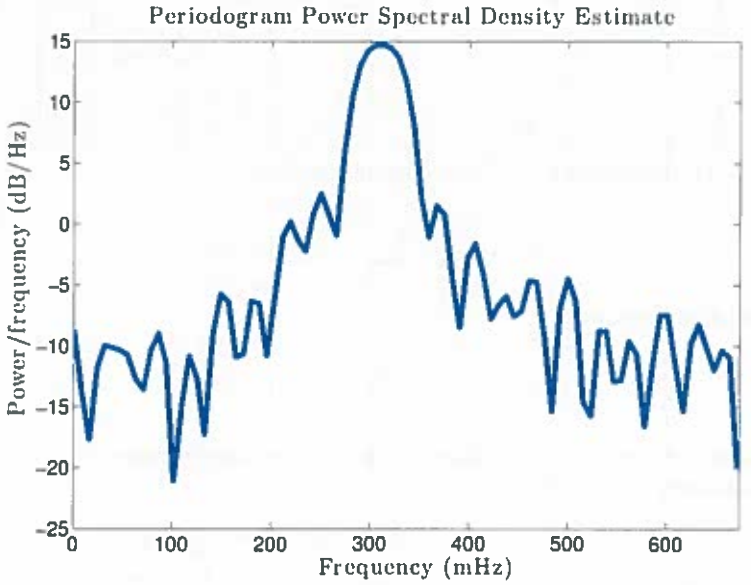


Figure 2: Periodogram of two complex exponentials with closely-spaced frequencies.

d) Familiarise yourself with the generation of complex exponential signals, and generate signals of different frequencies and length. Verify that by considering more data samples the periodogram starts showing the correct line spectra. [5]

e) Use the following code to find the desired line spectra using the MUSIC method.

```
[X,R] = corrmtx(x,14,'mod');
[S,F] = pmusic(R,2,[ ],1,'corr');
plot(F,S,'linewidth',2); set(gca,'xlim',[0.25 0.40]);
grid on; xlabel('Hz'); ylabel('Pseudospectrum');
```

Explain the operation of the first three lines in the code using the MATLAB documentation and the lecture notes. What is the meaning of the input arguments for the functions `corrmtx` and `pmusic`? Does the spectrum estimated using the MUSIC algorithm provide more detailed information? State briefly the advantages and disadvantages of the periodogram and the MUSIC algorithms and comment on the bias and variance. How accurate would a general spectrum estimate be when using MUSIC? [10]

2.2 Spectrum of Autoregressive Processes

In many spectrum estimation applications, only short data lengths are available; thus, classical spectrum estimation techniques based on the Fourier transform will not be able to resolve frequency elements spaced close to one another. In order to solve this problem, we can use modern spectrum estimation methods based on the pole-zero modelling of the data.

Consider a general ARMA(p, q) process given by

$$y(n) = a_1 y(n-1) + \cdots + a_p y(n-p) + w(n) + b_1 w(n-1) + \cdots + b_q w(n-q)$$

The power spectrum of y has the form

$$P_y(e^{j\omega}) = \frac{|\sum_{k=1}^{q} b_k e^{-jk\omega}|^2}{|1 - \sum_{k=1}^{p} a_k e^{-jk\omega}|^2}$$

Thus, the power spectrum can be estimated through the parameters $(a_1, ..., a_p, b_1, .., b_q)$. **The assumption of an underlying model for the data is the key difference between classical and modern spectrum estimation methods.**

For an AR process in particular, the power spectrum is the output of an all-pole filter given by

$$P_y(e^{j\omega}) = \frac{\sigma_w^2}{|1 - \sum_{k=1}^{p} a_k(k) e^{-jk\omega}|^2}$$

The parameters σ_w^2 and a $= \begin{bmatrix} a_1 & \dots & a_p \end{bmatrix}^T$ can be estimated by a set of $(p+1)$ linear equations

$$\begin{bmatrix} r_x(0) & r_x(1) & \cdots & r_x(p) \\ r_x(1) & r_x(0) & \cdots & r_x(p-1) \\ \vdots & \vdots & & \vdots \\ r_x(p) & r_x(p-1) & \cdots & r_x(0) \end{bmatrix} \begin{bmatrix} 1 \\ a_1 \\ \vdots \\ a_p \end{bmatrix} = \sigma_w^2 \begin{bmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

where $r_x(k)$ could be calculated using the biased autocorrelation estimate

$$r_x(k) = \frac{1}{N} \sum_{n=0}^{N-1-k} x(n+k)x(n)$$

or the unbiased autocorrelation estimate

$$r_x(k) = \frac{1}{N-k} \sum_{n=0}^{N-1-k} x(n+k)x(n)$$

a) Based on your answers in Section 2.1, elaborate on the shortcomings of using the unbiased ACF estimate when finding the AR parameters? [see Eq. (17)] [5]

b) Generate 1000 samples of data in MATLAB, according to the following equation [10]

$$x(n) = 2.76x(n-1) - 3.81x(n-2) + 2.65x(n-3) - 0.92x(n-4) + w(n)$$

where $w \sim \mathcal{N}(0, 1)$ and discard the first 500 samples (`x=x(500:end)`) to remove the transient output of the filter. Estimate the power spectrum density of the signal using model orders $p = 2, ..., 14$ and comment on the effects of increasing the order of the (assumed) underlying model by comparing the estimation to the true Power Spectral Density. Only plot the results of the model orders which produced the best results.

c) Repeat the experiment in b) for data length of 10,000 samples. What happens to the PSD when the chosen model order is lower (under-modelling) or higher (over-modelling) than the correct AR(4) model order? [5]

2.3 Principal Component Analysis

Load the file `PCAPCR.mat`[3] which include the data matrices $\mathbf{X} \in \mathbb{R}^{N \times d_x}$ and $\mathbf{Y} \in \mathbb{R}^{N \times d_y}$:

Training Data	Testing Data	Note
$\mathbf{X}$	-	Input variables, some of which are collinear. Each column represents N measurements of an input variable.
$\mathbf{X}_{\text{noise}}$	$\mathbf{X}_{\text{test}}$	Noise corrupted input matrices $\mathbf{X}_{\text{noise}} = \mathbf{X} + \mathbf{N}_{\mathbf{X}}$, where elements of $\mathbf{N}_{\mathbf{X}}$ were drawn from a zero-mean Gaussian distribution.
$\mathbf{Y}$	$\mathbf{Y}_{\text{test}}$	Output variables obtained from $\mathbf{Y} = \mathbf{X}\mathbf{B} + \mathbf{N}_{\mathbf{Y}}$ where $\mathbf{B}$ is unknown. Each column represents N measurements of an output variable. Elements of $\mathbf{N}_{\mathbf{Y}}$ were drawn from a zero-mean Gaussian distribution.

Using the Matlab command `svd`, obtain the singular value decomposition for $\mathbf{X}$ and $\mathbf{X}_{\text{noise}}$.

a) Plot the singular values of $\mathbf{X}$ and $\mathbf{X}_{\text{noise}}$ (hint: use the `stem` command). Identify the rank of the data $\mathbf{X}$. Plot the square error between each singular value of $\mathbf{X}$ and $\mathbf{X}_{\text{noise}}$. What affect does the noise have on the singular values? At what point would it become hard to identify the rank of the matrix $\mathbf{X}_{\text{noise}}$? [5]

b) Using only the r significant principal components (as determined by the identified rank), create a low-rank approximation of $\mathbf{X}_{\text{noise}}$, denoted $\tilde{\mathbf{X}}_{\text{noise}}$. Compare the difference (error) between the variables (columns) of the noiseless input matrix $\mathbf{X}$ and the noise corrupted matrix $\mathbf{X}_{\text{noise}}$ and denoised matrix $\tilde{\mathbf{X}}_{\text{noise}}$. [5]

The ordinary least squares (OLS) estimate for the unknown regression matrix $\mathbf{B}$, is given

$$\hat{\mathbf{B}}_{\text{OLS}} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{Y} \tag{18}$$

Since the matrix $\mathbf{X}^T\mathbf{X}$ from the original data $\mathbf{X}$ is sub-rank, the OLS solution in (18) becomes intractable. On the other hand, using the noisy data $\mathbf{X}_{\text{noise}}$ admits the OLS solution as $\mathbf{X}_{\text{noise}}^T\mathbf{X}_{\text{noise}}$ is full-rank but may introduce spurious correlations in calculating the regression coefficients.

To circumvent this, the principal component regression (PCR) applies principal component analysis (PCA) on the input matrix $\mathbf{X}_{\text{noise}}$ to compute the estimate of $\mathbf{B}$. Specifically, the SVD of $\mathbf{X}_{\text{noise}}$ is given by $\mathbf{X}_{\text{noise}} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$. By retaining the r principal components (r-singular values and the associated singular vectors), the PCR solution avoids both the problem of collinearity and noise in the input matrix $\mathbf{X}$. The PCR solution is given by

$$\hat{\mathbf{B}}_{\text{PCR}} = \mathbf{V}_{1:r}(\mathbf{\Sigma}_{1:r})^{-1}\mathbf{U}_{1:r}^T\mathbf{Y}$$

where the subscript $(1:r)$ denotes the r-largest singular values and the corresponding singular vectors.

c) Calculate the OLS and PCR solutions between $\mathbf{X}_{\text{noise}}$ and $\mathbf{Y}$. Next, compare the estimation error between $\mathbf{Y}$ and $\hat{\mathbf{Y}}_{\text{OLS}} = \mathbf{X}_{\text{noise}}\hat{\mathbf{B}}_{\text{OLS}}$ and $\hat{\mathbf{Y}}_{\text{PCR}} = \tilde{\mathbf{X}}_{\text{noise}}\hat{\mathbf{B}}_{\text{PCR}}$. What happens when you use try to estimate the data from the test-set using the regression coefficients computed from the training set? Quantify this performance by comparing $\mathbf{Y}_{\text{test}}$ and $\hat{\mathbf{Y}}_{\text{test-OLS}} = \mathbf{X}_{\text{test}}\hat{\mathbf{B}}_{\text{OLS}}$ and $\hat{\mathbf{Y}}_{\text{test-PCR}} = \tilde{\mathbf{X}}_{\text{test}}\hat{\mathbf{B}}_{\text{PCR}}$. [5]

d) The best way to assess the effectiveness of the PCR compared to the OLS solution is by testing the estimated regressions coefficients, $\hat{\mathbf{B}}$, over an ensemble of test data. We have provided you with a script that outputs a new realisation of the test data, $\mathbf{Y}$, and its estimate, $\hat{\mathbf{Y}}$, with the regression coefficients you provide the function with: [5]

$$[\hat{\mathbf{Y}}, \mathbf{Y}] = \texttt{regval}(\hat{\mathbf{B}}).$$

Using the *same* PCR and OLS regression coefficients, from (c), compute and compare the mean square error estimate, $\text{MSE} = E\{\|\mathbf{Y} - \hat{\mathbf{Y}}\|_F^2\}$ for the PCR and OLS schemes from the realisations of $\mathbf{Y}$ and $\hat{\mathbf{Y}}$ provided by the function `regval`.

2.4 Real World Signals: Respiratory Sinus Arrhythmia from RR-Intervals

Note: Use the RRI data from your experiment.

Respiratory sinus arrhythmia (RSA) refers to the modulation of cardiac function by respiratory effort. This can be readily observed by the speeding up of heart rate during inspiration ("breathing in") and the slowing down of heart rate during expiration ("breathing out"). The strength of RSA in an individual can be used to assess cardiovascular health. Breathing at regular rates will highlight the presence of RSA in the cardiac (ECG) data.

a) Apply the standard periodogram as well as the averaged periodogram with different window lengths (e.g. 50 s, 150 s) to obtain the power spectral density of the RRI data. Plot the PSDs of the RRI data obtained from the three trials separately. [10]

b) Explain the differences between the PSD estimates of the RRI data from the three trials? Can you identify the peaks in the spectrum corresponding to frequencies of respiration for the three experiments? [5]

c) Plot the AR spectrum estimate for the RRI signals for the three trials[4]. To find the optimal AR model order, experiment with your model order until you observe a peak in the spectrum (approximately) corresponding to the theoretical respiration rate. List the differences you observe between your estimated AR spectrum and the periodogram estimate in Part a). [10]

[3]Download the PCR files from: `http://www.commsp.ee.ic.ac.uk/~mandic/PCR.zip`

[4]Use the MATLAB function `aryule` to estimate the AR coefficients for your RRI signal.

3 Adaptive signal processing

Aims: Students will learn to:

- Implement adaptive filtering algorithms and identify the appropriate configurations for different applications.
- Explain and demonstrate the effects of the various algorithm parameters on the performance of adaptive filters.
- Derive theoretical performance measures of the adaptive algorithms and compare them to experimental results.

3.1 The Least Mean Square (LMS) Algorithm

To study nonstationary signals, adaptive filters that approximate, in a recursive fashion, the Wiener solution are considered. The simplest adaptive filter is based on the least mean square (LMS) algorithm, which recursively updates the weights using a gradient-based minimisation of the (instantaneous error) cost function $J(n) = \frac{1}{2}e^2(n)$ according to

$$\mathbf{w}(n+1) = \mathbf{w}(n) + \mu e(n)\mathbf{x}(n), \qquad n = 0, 1, \ldots \tag{19}$$

where $\mu > 0$ is the adaptation gain which controls the stability of the algorithm, and $\mathbf{w}(0)$ can be set to any value (usually $\mathbf{w}(0) = \mathbf{0}$). Notice that the time-varying weight vector of the LMS filter $\mathbf{w}(n)$ allows for the estimation of non-stationary signals.

The error $e(n) = x(n) - \hat{x}(n)$ is calculated as the difference between the original process $x(n)$ and the estimate $\hat{x}(n)$, given by

$$\hat{x}(n) \quad = \quad \mathbf{w}^T(n)\mathbf{x}(n) = \sum_{m=1}^{M} w_m(n)x(n-m). \tag{20}$$

Note: $x(n)$, $\hat{x}(n)$ and $e(n)$ are scalar quantities, whereas $\mathbf{w}(n)$ and $\mathbf{x}(n)$ are $M \times 1$ column vectors.

Identification of AR processes. Linear adaptive filters have a wide range of applications. Speech denoising, for instance, involves non-stationary signals, and therefore requires adaptive estimation. Consider the structure shown in Figure 3 consisting of two separate stages: the **synthesis** of a signal from white noise using an autoregressive model, and the **analysis** stage which performs the adaptation of coefficients to recreate the original signal. We now consider the problem

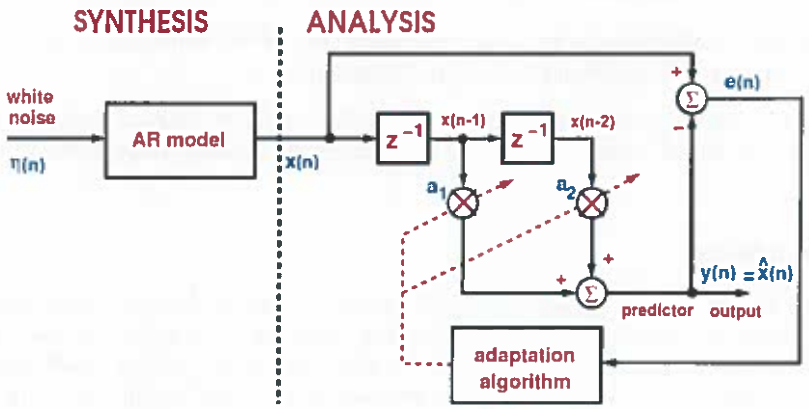


Figure 3: Synthesis and analysis structure for an AR model.

of adaptive linear prediction using the LMS algorithm. Let $x(n)$ be a second-order auto-regressive process satisfying the difference equation

$$x(n) = a_1 x(n-1) + a_2 x(n-2) + \eta(n) \tag{21}$$

where $\eta(n) \sim \mathcal{N}(0, \sigma_\eta^2)$. Your aim is to implement an LMS estimator of the AR coefficients a_1 and a_2 using the signal $x(n)$.

a) The LMS filter can be seen as a dynamical system for which the inputs are $x(n-1), x(n-2)$ and the outputs represent the estimates $\hat{a}_1$, $\hat{a}_2$ and $\hat{x}(n)$. If the original parameters are $a_1 = 0.1$, $a_2 = 0.8$ and $\sigma_\eta^2 = 0.25$, what are the entries of the correlation matrix of the input vector $\mathbf{x}(n) = [x(n-1), x(n-2)]^T$? (*Hint: It will be* 2×2 *matrix.*) Furthermore, for what range of values of the step size μ will the LMS converge in the mean? [5]

b) Implement an LMS adaptive predictor using $N = 1000$ samples of $x(n)$ as in a) and plot the squared prediction error $e^2(n)$ dB i.e. $10\log(e^2(n))$ along time using step sizes $\mu = 0.05$ and $\mu = 0.01$. Repeat this experiment for 100 different realizations of $x(n)$ and plot the learning curve by averaging the plots of $10\log(e^2(n))$. [5]

The LMS weight estimates fluctuate around the optimal solution and these fluctuations introduce **excess error** to the minimum achievable error signal. The additional error power introduced by the adaptive filter can be quantified by the excess mean square error (EMSE) metric which measures the difference between the mean-square error introduced by adaptive filters and the minimum attainable mean square error of a Wiener filter. The corresponding misadjustment, $\mathcal{M}$, is the ratio of the excess mean square error and the minimum mean square error

$$\text{MSE} = \lim_{n\to\infty} \mathbb{E}\{e^2(n)\} = \sigma_\eta^2 + \text{EMSE} \tag{22}$$

$$\mathcal{M} = \frac{\text{EMSE}}{\sigma_\eta^2}. \tag{23}$$

For small step-sizes, the misadjustment of the LMS is approximated as

$$\mathcal{M}_{\text{LMS}} \approx \frac{\mu}{2}\text{Tr}\{\mathbf{R}\} \tag{24}$$

where $\mathbf{R}$ is the auto-correlation matrix of the input, μ is the learning rate and $\text{Tr}\{\cdot\}$ denotes the matrix trace operator.

c) For the example in Part a), estimate the corresponding misadjustment by time-averaging over the steady state of the ensemble-averaged learning curves using 100 independent trials of the experiment. Compare the estimated values with the theoretical LMS misadjustment in (24). [5]

d) Estimate the steady state values of the adaptive filter coefficients for the step-sizes of $\mu = 0.05$ and $\mu = 0.01$. Note that you may do this by averaging the steady-state values of the coefficients (obtained along the the final iterations of the LMS) over 100 independent trials of the experiment. Compare these estimated values with the true coefficients, and explain your findings. [5]

The Leaky LMS Algorithm

When the input signal $\mathbf{x}(n)$ has an autocorrelation matrix with zero eigenvalues, the LMS adaptive filter has one or more weights that do not converge and this may lead to instability. One way to stabilize the algorithm is to introduce a *leakage* coefficient, $0 < \gamma < 1$, to force the filter weights to converge.

$$\text{Leaky LMS:} \quad \mathbf{w}(n+1) = (1 - \mu\gamma)\,\mathbf{w}(n) + \mu e(n)\mathbf{x}(n) \tag{25}$$

e) Derive the LMS coefficient update for $\mathbf{w}(n)$ that minimizes the cost function $J_2(n) = \frac{1}{2}\left(e^2(n) + \gamma\|\mathbf{w}(n)\|_2^2\right)$ and show that you obtain the leaky LMS equation shown in Equation (25). [5]

f) Implement the leaky LMS algorithm (with different values for γ and μ) to estimate the AR coefficients of the signal given in Part a). Why do the weights of the leaky LMS algorithm converge to incorrect values for the parameters a_1 and a_2? [5]

3.2 Adaptive Step Sizes

Choosing the step-size for the LMS is a challenging task due to the trade-off between convergence speed and steady state error variance. Ideally, the step-size should be large in the beginning of the adaptation and small as the parameter estimates become closer the true value, thus allowing for fast convergence and reduced steady-state error. To this end, variable step-size (VSS) algorithms aim to solve two simultaneous unconstrained optimisation procedures – to minimise the cost function with respect to the weight vector, while at the same time minimising the cost function with respect to the step size. Consider the LMS algorithm

$$\mathbf{w}(n+1) = \mathbf{w}(n) + \mu(n)e(n)\mathbf{x}(n) \tag{26}$$

where the learning rate $\mu(n)$ is made adaptive using gradient adaptive step-size (GASS) algorithms based on the minimisation of $\nabla_\mu J$, such as the algorithms introduced by Benveniste [1], Ang & Farhang [2], and Matthews & Xie [3]. Within GASS, the learning rate $\mu(n)$ is updated according to

$$\mu(n+1) = \mu(n) + \rho e(n)\mathbf{x}^T(n)\psi(n) \tag{27}$$

where the term $\psi(n)$ can take one of the following forms

$$\text{Benveniste}: \psi(n) = \left[\mathbf{I} - \mu(n-1)\mathbf{x}(n-1)\mathbf{x}^T(n-1)\right]\psi(n-1) + e(n-1)\mathbf{x}(n-1) \tag{28}$$

$$\text{Ang \& Farhang}: \psi(n) = \alpha\psi(n-1) + e(n-1)\mathbf{x}(n-1), \qquad 0 < \alpha < 1 \tag{29}$$

$$\text{Matthews \& Xie}: \psi(n) = e(n-1)\mathbf{x}(n-1) \tag{30}$$

a) Implement the three GASS algorithms above, and compare their performance when identifying a real-valued MA(1) system [15]

$$x(n) = 0.9\eta(n-1) + \eta(n) \qquad \eta \sim \mathcal{N}(0, 0.5). \tag{31}$$

Explain the performance advantages/disadvantages against a standard LMS algorithm (with a fixed step size, $\mu = 0.01$ and $\mu = 0.1$). Plot the weight error curves ($\tilde{w}(n) = w_o - w(n)$ where $w_o = 0.9$) for each algorithm and comment on their convergence speed and steady state error.

Since the weight update of the LMS is proportional to the input vector $\mathbf{x}(n)$, it is often useful to normalise the step size by using the normalized LMS (NLMS) algorithm in the form

$$\mathbf{w}(n+1) = \mathbf{w}(n) + \frac{\beta}{\epsilon + \|\mathbf{x}(n)\|^2} e(n)\mathbf{x}(n) = \mathbf{w}(n) + \frac{\beta}{\epsilon + \mathbf{x}^T(n)\mathbf{x}(n)} e(n)\mathbf{x}(n) \tag{32}$$

where the regularization factor ϵ prevents the algorithm from becoming unstable if the input vector is zero.

b) Verify that the update equation [5]

$$\mathbf{w}(n+1) = \mathbf{w}(n) + \mu e_p(n)\mathbf{x}(n) \tag{33}$$

based upon the *a posteriori* error $e_p(n) = d(n) - \mathbf{x}^T(n)\mathbf{w}(n+1)$ is equivalent to the NLMS algorithm. Comment upon the relationship between β, ϵ and μ.

Hint: Start by expressing $\Delta\mathbf{w}(n) = \mathbf{w}(n+1) - \mathbf{w}(n)$ *in terms of the a priori error* $e(n)$ *as*

$$\Delta\mathbf{w}(n) = \mu e(n) \left[1 - \mu \frac{\|\mathbf{x}(n)\|^2}{1 + \mu\|\mathbf{x}(n)\|^2} \right] \mathbf{x}(n) \tag{34}$$

The gain of the NLMS algorithm can be made adaptive by making the regularization factor $\epsilon \to \epsilon(n)$ time varying. The generalized normalized gradient descent (GNGD) algorithm [4] adapts $\epsilon(n)$ via a gradient method, and is given by

$$\epsilon(n+1) = \epsilon(n) - \rho\mu \frac{e(n)e(n-1)\mathbf{x}^T(n)\mathbf{x}(n-1)}{(\epsilon(n-1) + \|\mathbf{x}(n-1)\|^2)^2} \tag{35}$$

c) Implement the GNGD algorithm for the system identification problem in part (a), plot the weight estimates and compare it with Benveniste's algorithm. Compare the computational complexity (number of multiplications and additions every time instant n) of the Benveniste's GASS algorithm to the GNGD. [10]

3.3 Adaptive Noise Cancellation

This exercise deals with noise suppression for two de-noising adaptive filtering configurations.

- **Adaptive Line Enhancer (ALE).** An ALE consists of a delay operator and a linear predictor, as illustrated in Fig 4. The noise-corrupted signal $s(n)$ is composed of a clean signal $x(n)$ and a noise term (not necessarily white), $\eta(n)$. The delay, Δ must be chosen such that noise in the signal $s(n)$ and the predictor input $\mathbf{u}(n) = [s(n-\Delta), \ldots, s(n-\Delta-M+1)]^T$ are uncorrelated, so that only the noise (and not the signal $x(n)$) is suppressed by the linear predictor. This linear predictor is an FIR filter and its weights are adjusted using an adaptive algorithm, usually the LMS algorithm.

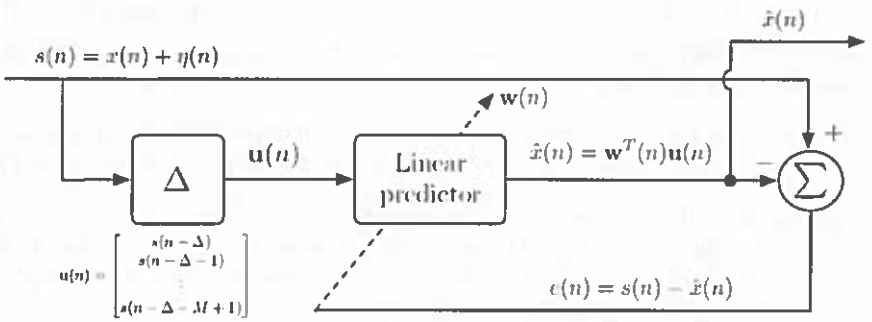


Figure 4: Adaptive line enhancement configuration.

- **Adaptive Noise Cancellation (ANC).** This configuration receives the noise-corrupted primary input $s(n)$, which contains the corrupted signal, and a secondary noise input $\epsilon(n)$, which is correlated in some unknown way with the primary noise. For example, in a mobile phone, the primary input comes from the microphone that the user talks into and the secondary input may come from an external microphone used to pick-up the ambient noise. A linear predictor is used to estimate the noise present in the signal, $\eta(n)$, using the secondary noise measurement $\epsilon(n)$ and then the clean signal $\hat{x}(n)$ is formed by subtracting $\hat{\eta}(n)$ from $s(n)$. The ANC configuration is shown in Fig 5.

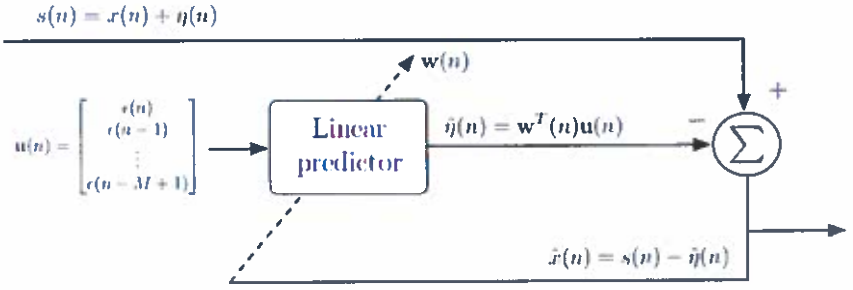


Figure 5: Adaptive noise cancellation configuration.

For both the noise suppression configurations, the prediction error $\varepsilon(n)$ is defined as the difference between the clean signal, $x(n)$ and the estimated de-noised signal $\hat{x}(n)$. The estimate of the mean square prediction error (MSPE) is

$$\text{MSPE} = \frac{1}{N}\sum_{n=0}^{N-1}\varepsilon^2(n) = \frac{1}{N}\sum_{n=0}^{N-1}\big(x(n) - \hat{x}(n)\big)^2. \tag{36}$$

Generate a noise-corrupted signal

$$s(n) = x(n) + \eta(n) \tag{37}$$

where $x(n)$ is a unit amplitude sinusoid of angular frequency $\omega_o = 0.01\pi$ and $\eta(n)$ is coloured noise generated using the moving average filter

$$\eta(n) = v(n) + 0.5v(n-2) \tag{38}$$

where $v(n)$ is white noise with unit variance.

a) Given the model for the noise in (38), what is the minimum value for the delay Δ that may be used in the adaptive line enhancer in Fig 4 for a filter length of $M > 1$? *Hint: Start by expanding the Mean Square Error* $\mathbb{E}\{(s(n) - \hat{x}(n))^2\}$ *to determine how the correlation in the noise affects the linear predictor.* [10]

   Write a MATLAB program for the adaptive line enhancement using the LMS algorithm and justify your choice of minimum Δ.

b) Generate 1000 samples of $s(n)$ and use your MATLAB program to estimate $x(n)$ for filter orders $M = 5, 10, 15, 20$. Use values for Δ that range from the minimum value determined in part (a) to $\Delta = 25$. What is the dependence (if any) between the delay Δ and the MSPE (i.e. how well the de-noising algorithm has performed)? [10]

   What is the effect of the filter order M on the MSPE? Given that the computational cost increases with M, what filter order would be a good pragmatic choice?

c) Write a MATLAB program for the adaptive noise cancellation configuration (see Fig 5). Use the MSPE measure in (36) to compare the relative performance of the ANC and ALE in de-noising the noisy sinusoid in (37). [5]

d) Load a single-channel EEG Data[5] (either `Cz` or `POz`) from the file `EEG_Data_Assignment2.mat`. As seen in the previous assignments, there is a strong 50 Hz component introduced by the mains. Using the ANC configuration, remove the 50 Hz component by generating a synthetic reference input composed of a sinusoid of 50 Hz corrupted by white Gaussian noise. (*Hint: Experiment with different step-sizes, μ and filter lengths, M, to suppress the 50 Hz component without affecting the other frequency components*). Plot the spectrum for the corrupted and denoised EEG data using the `spectrogram` function with appropriate window length and overlap. What do you observe? [15]

[5] Download the EEG data from `http://www.commsp.ee.ic.ac.uk/~mandic/EEG_Data.zip`

4 Widely Linear Filtering and Adaptive Spectrum Estimation

Aims: Students will learn to:

- Implement widely linear adaptive filtering configurations for the estimation of noncircular signals.
- Use widely linear modelling in renewable energy and smart grid applications.
- Use adaptive filters for spectral estimation of non-stationary data.

4.1 Complex LMS and Widely Linear Modelling

Complex-valued signals play a pivotal role in communications, array signal processing, biomedical signal processing and related fields. These signals can be either complex by design, such as symbols used in data communication, or are made complex by convenience of representation e.g. 2D wind speed and direction. The complex LMS (CLMS) is one of the most widely used estimation algorithms to process complex valued data and is given by

$$\begin{aligned} y(n) &= \mathbf{h}^H(n)\mathbf{x}(n) \\ e(n) &= y(n) - \hat{y}(n) \\ \mathbf{h}(n+1) &= \mathbf{h}(n) + \mu e^*(n)\mathbf{x}(n) \end{aligned} \tag{39}$$

where μ is the learning rate. The CLMS is similar, in form, to its real-valued counterpart, (19), used in Part 3.

Although statistical tools for complex random variables are traditionally treated as generic extensions of their real-valued counterparts, this approach is valid only for circular random variables. It has now been accepted that the standard strictly linear model for complex data is not guaranteed to capture the complete second-order statistical relationship between the input and the output as generic strictly linear extensions of real-valued estimators cater only for circular data i.e. data with rotation invariant probability distributions.

To account for general complex data, we have to employ the widely linear framework. By using an augmented input vector, $\mathbf{x}^a = [\mathbf{x}^T \ \mathbf{x}^H]^T$, that contains the input vector and its conjugate, the widely linear model is equipped with the sufficient degrees of freedom to fully exploit the second order statistics in the data. The Augmented CLMS (ACLMS) [5] is given by

$$\begin{aligned} \hat{y}(n) &= \mathbf{h}^H(n)\mathbf{x}(n) + \mathbf{g}^H(n)\mathbf{x}^*(n) \\ e(n) &= y(n) - \hat{y}(n) \\ \mathbf{h}(n+1) &= \mathbf{h}(n) + \mu e^*(n)\mathbf{x}(n) \\ \mathbf{g}(n+1) &= \mathbf{g}(n) + \mu e^*(n)\mathbf{x}^*(n). \end{aligned} \tag{40}$$

a) Generate a first-order widely-linear-moving-average process, WLMA(1), driven by circular white Gaussian noise, $x(n)$ [10]

$$y(n) = x(n) + b_1 x(n-1) + b_2 x^*(n-1) \qquad x \sim \mathcal{N}(0,1) \tag{41}$$

where $b_1 = 1.5 + 1j$ and $b_2 = 2.5 - 0.5j$. Write a MATLAB function for the ACLMS and implement both the CLMS and ACLMS in the system identification setting to identify the WLMA model in (41). Plot the learning curve, $10\log|e(n)|^2$, for the ACLMS and CLMS. Comment on the steady state error of the ACLMS and CLMS.

Hint: To obtain a smoother learning curve, plot the ensemble average of the learning curve from 100 independent simulations.

b) Load the bivariate wind data[6] of the wind speeds in the East-West direction, v_{east}, and North-South direction, v_{north}. Form a complex-valued wind signal [10]

$$v[n] = v_{\text{east}}[n] + jv_{\text{north}}[n] \tag{42}$$

for the three wind regimes (low, medium, high). Use the `scatter(x,y)` function to plot a scatter diagram (the scatter diagram of the real and imaginary parts of a signal is also referred to as a circularity plot) for these three regimes. Comment on the circularity of the complex wind signal for low, medium and high wind speeds. Configure the CLMS and ACLMS filters in a prediction setting (see Equation (20)) to perform a one-step ahead prediction of the complex wind data. Experiment with different filter lengths and comment on which algorithm (CLMS or ACLMS) performs better for the different wind regimes.

[6] Download the wind data from `http://www.commsp.ee.ic.ac.uk/~mandic/wind_dataset.zip`

Frequency Estimation in Three Phase Power Systems

Consider the problem of estimating the nominal frequency of a three phase power system (50 Hz in the UK, 60 Hz in the US). The three-phase voltages of a power system measured at a single node (in a noiseless environment) can be represented by a vector

$$\begin{bmatrix} v_a(n) \\ v_b(n) \\ v_c(n) \end{bmatrix} = \begin{bmatrix} V_a(n)\cos(2\pi\frac{f_\circ}{f_s}n + \phi) \\ V_b(n)\cos(2\pi\frac{f_\circ}{f_s}n + \phi + \Delta_b - \frac{2\pi}{3}) \\ V_c(n)\cos(2\pi\frac{f_\circ}{f_s}n + \phi + \Delta_c + \frac{2\pi}{3}) \end{bmatrix} \tag{43}$$

where the magnitudes $V_a(n)$, $V_b(n)$, and $V_c(n)$ are the peak values of these sinusoids, ϕ is the phase shift, k is the time index, where $f_\circ$ is the system frequency and f_s is the sampling frequency. The terms Δ_b, Δ_c are used to indicate the phase distortions relative to the balanced three-phase system. A power system is said to be in a balanced operating condition when: (i) All the three signals have the same peak voltage, i.e. $V_a(n) = V_b(n) = V_c(n) = V$, and (ii) the three voltages have a phase shift of exactly 120 degrees, i.e. $\Delta_b = \Delta_c = 0$.

The three-phase system can be projected onto two orthogonal axes, forming the I and Q components $v_\alpha(n)$ and $v_\beta(n)$, using the Clarke Transform (also known as the $\alpha\beta$-transform)

$$\begin{bmatrix} v_0(n) \\ v_\alpha(n) \\ v_\beta(n) \end{bmatrix} = \underbrace{\sqrt{\frac{2}{3}}\begin{bmatrix} \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \\ 1 & -\frac{1}{2} & -\frac{1}{2} \\ 0 & \frac{\sqrt{3}}{2} & -\frac{\sqrt{3}}{2} \end{bmatrix}}_{\text{C=Clarke Matrix}} \begin{bmatrix} v_a(n) \\ v_b(n) \\ v_c(n) \end{bmatrix} \tag{44}$$

The third component, known as the zero-sequence voltage, $v_0(n)$, is needed to make the transformation in (44) invertible. Under a balanced system condition, the transformation is designed to give $v_0(n) = 0$.

The α and β components can be conveniently represented as a complex Clarke voltage $v(n) = v_\alpha(n) + jv_\beta(n)$. Under balanced conditions, the zero-sequence voltage v_0 is equal to zero and the complex voltage is a complex exponential

$$v(n) = \sqrt{\frac{3}{2}}Ve^{j\left(2\pi\frac{f_\circ}{f_s}n+\phi\right)} \tag{45}$$

When the system is unbalanced, the complex-valued representation in (45) is incorrect, and the general form of the complex-valued voltage that arises from the Clarke Transform is instead

$$v(n) = A(n)e^{j\left(2\pi\frac{f_\circ}{f_s}n+\phi\right)} + B(n)e^{-j\left(2\pi\frac{f_\circ}{f_s}n+\phi\right)} \tag{46}$$

where $A(n) = \frac{\sqrt{6}}{6}\left(V_a(n) + V_b(n)e^{j\Delta_b} + V_c(n)e^{j\Delta_c}\right)$ and $B(n) = \frac{\sqrt{6}}{6}\left(V_a(n) + V_b(n)e^{-j(\Delta_b+2\pi/3)} + V_c(n)e^{-j(\Delta_c-2\pi/3)}\right)$. The coefficient $B(n)$ becomes zero when the system is balanced $\left(V_a(n) = V_b(n) = V_c(n)\right.$ and $\left.\Delta_b = \Delta_c = 0\right)$ and this model collapses into that shown in (45).

c) Generate two sets of complex voltages, one balanced and one unbalanced. To generate an unbalanced system, change the magnitude and/or phase of one or more phases. Plot the circularity diagrams of these complex $\alpha - \beta$ voltages. Comment on the shape of the circularity diagram when the system is balanced vs. unbalanced. How would you use the circularity diagram to identify a fault in the system? [5]

d) Consider the strictly linear and widely linear autoregressive models of order 1, given by [10]

$$\text{Strictly Linear:} \quad v(n+1) = h^*(n)v(n) \tag{47}$$

$$\text{Widely Linear:} \quad v(n+1) = h^*(n)v(n) + g^*(n)v^*(n) \tag{48}$$

Show that the frequency of the balanced complex $\alpha - \beta$ voltage in (45) can be derived from the coefficients $h(n)$ in (47) as

$$f_\circ(n) = \frac{f_s}{2\pi}\arctan\left\{\frac{\Im\{h(n)\}}{\Re\{h(n)\}}\right\} \tag{49}$$

and the frequency of the unbalanced voltage in (46) can be derived from the coefficients $h(n)$ and $g(n)$ in (48) as

$$f_\circ(n) = \frac{f_s}{2\pi}\arctan\left\{\frac{\sqrt{\Im^2\{h(n)\} - |g(n)|^2}}{\Re\{h(n)\}}\right\} \tag{50}$$

e) Use the CLMS given (39) and ACLMS algorithms given in (40) to estimate the frequency of the $\alpha - \beta$ voltages you generated in Part b). For unbalanced system voltages, does the CLMS give the correct frequency estimate? If not, why? [15]

4.2 Adaptive AR Model Based Time-Frequency Estimation

In Part 2.2, the AR spectrum was estimated by assuming stationary signals. Therefore, the block-based estimate of the AR coefficients (and the corresponding power spectrum) will be inaccurate when the signal is non-stationary. In this section, the concepts of non-stationary (online) spectrum estimation will be explored.

a) Generate the frequency modulated (FM) signal $y(n) = e^{j\left(\frac{2\pi}{f_s}\phi(n)\right)} + \eta(n)$ where $\eta(n)$ is circular complex-valued white noise with zero mean and variance $\sigma_\eta^2 = 0.05$ and the phase $\phi(n) = \int f(n)\, dn$ is generated as[7] [10]

$$f(n) = \frac{d\phi(n)}{dn} = \begin{cases} 100, & 1 \le n \le 500 \\ 100 + \frac{n-500}{2}, & 501 \le n \le 1000 \\ 100 + \left(\frac{n-1000}{25}\right)^2, & 1001 \le n \le 1500 \end{cases} \tag{51}$$

Use the `aryule` function find the AR(1) coefficient for the complete signal of length 1500, then plot the power spectrum of the signal.

Note: The function `[h,w] = freqz(1,a,N)` returns the N-point frequency response vector, h, and the corresponding angular frequency vector, w, for the digital filter with AR coefficients in the vector a. Does this method capture the changes in frequency given by Equation (51)?

b) Implement the CLMS algorithm to estimate the AR coefficient of the signal $y(n)$, see (47). At each time instant, compute the frequency spectrum of the signal using the `freqz` function with the coefficient estimates from the CLMS. Plot the time-frequency spectrum. (*Hint: Use the code below.*) Comment on the CLMS based spectrum estimate implemented in this part, compared to the stationary AR spectrum in Part a). [10]

```
for n = 1:N
    % Run complex-valued LMS algorithm to estimate AR coefficient â_1(n)
    [h ,w]= freqz(1 , [1; -â_1^*(n)], 1024);  % Compute power spectrum
    H(:, n) = abs(h).^2;                      % Store it in a matrix
end
% Remove outliers in the matrix H
medianH = 50*median(median(H));
H(H > medianH) = medianH;
% Plot time-frequency diagram
```

(*Hint: To plot a time-frequency diagram, first plot a 3D graph with the entries of* `H` *in the z-axis, time instant in the x-axis, frequency* `w` *in the y-axis. Then, use the* `view(2)` *command to obtain a 2D color-heat map.*)

4.3 A Real Time Spectrum Analyser Using Least Mean Square

Consider estimating a signal $y(n)$ using a linear combination of N harmonically related sinusoids[8] as

$$\begin{aligned} \hat{y}(n) &= \sum_{k=0}^{N-1} w(k) e^{j2\pi kn/N} \\ &= w(0) + w(1)e^{j2\pi n/N} + w(2)e^{j(2\pi n/N)2} + \cdots + w(N-1)e^{j(2\pi n/N)(N-1)} \end{aligned}$$

where $\hat{y}(n)$ is the *estimate* of the signal $y(n)$ and $w(k)$ are the unknown weights (in this case, the Fourier coefficients) that need to be estimated. Collecting all N estimates of the signal $y(n)$ into vector allows us to express this problem as

$$\underbrace{\begin{bmatrix} \hat{y}(0) \\ \hat{y}(1) \\ \vdots \\ \hat{y}(N-1) \end{bmatrix}}_{\hat{\mathbf{y}}} = \underbrace{\begin{bmatrix} 1 & 1 & \cdots & 1 \\ 1 & e^{j\frac{2\pi}{N}(1)(1)} & \cdots & e^{j\frac{2\pi}{N}(1)(N-1)} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & e^{j\frac{2\pi}{N}(N-1)(1)} & \cdots & e^{j\frac{2\pi}{N}(N-1)(N-1)} \end{bmatrix}}_{\mathbf{F}} \underbrace{\begin{bmatrix} w(0) \\ w(1) \\ \vdots \\ w(N-1) \end{bmatrix}}_{\mathbf{w}}$$

Within the least squares framework, the vector $\mathbf{w}$ is found by minimising the sum of squared errors between the estimated signal $\hat{y}(n)$ and true signal $y(n)$. This least squares problem is solved by minimising the cost function

$$\min_{\mathbf{w}} \|\mathbf{y} - \hat{\mathbf{y}}\|^2 = \min_{\mathbf{w}} \sum_{n=0}^{N-1} |y(n) - \hat{y}(n)|^2 \tag{52}$$

where $\mathbf{y}$ is the vector containing the actual data $\mathbf{y} = \begin{bmatrix} y(0) & y(1) & \ldots & y(N-1) \end{bmatrix}^T$.

[7] First generate the frequency in (51), then integrate it using the MATLAB function `cumsum` to obtain the phase $\phi(n)$.

[8] Each sinusoid is a multiple of the frequency $\frac{f_s}{N}$.

a) Show that the least squares solution for the problem in (52) is given by $\mathbf{w} = \left(\mathbf{F}^H\mathbf{F}\right)^{-1}\mathbf{F}^H\mathbf{y}$ and comment on its relationship to the discrete Fourier transform (DFT) formula. [5]

b) Given the least squares interpretation of the DFT, in your own words, explain the Fourier transform in terms of the change of basis and projections. [5]

As seen in Part 4.2, a block-based solution can be implemented in an online fashion using adaptive filters. Similarly, treating the DFT as a least squares solution allows us to implement an adaptive version using the CLMS where the desired signal to the CLMS can be treated as the time-domain signal to be Fourier transformed and the input to the filter at time n is the complex phasor

$$\mathbf{x}(n) = \frac{1}{N}\begin{bmatrix} 1 & e^{j\frac{2n\pi}{N}} & e^{j\frac{4n\pi}{N}} & \cdots & e^{j\frac{2n(N-1)\pi}{N}} \end{bmatrix}^T \tag{53}$$

The filter weight vector $\mathbf{w}(n) = \begin{bmatrix} w_0(n) & w_1(n) & \cdots & w_{N-1}(n) \end{bmatrix}^T$ is adapted using the CLMS algorithm (see Figure 6)

$$\begin{aligned} \hat{y}(n) &= \mathbf{w}^H(n)\mathbf{x}(n) \\ e(n) &= y(n) - \hat{y}(n) \\ \mathbf{w}(n+1) &= \mathbf{w}(n) + \mu e^*(n)\mathbf{x}(n) \end{aligned} \tag{54}$$

Widrow et al. [6] showed that by setting the learning rate $\mu = 1$, and iterating over n from the initial conditions $\mathbf{w}(0) = \mathbf{0}$, the weight vectors $\mathbf{w}(n)$ converge to the DFT solution.

c) Implement the DFT-CLMS algorithm given in (54) for the frequency modulated signal from Part 4.2 a). Plot the magnitude of the weight vector $\mathbf{w}(n)$ at every time instant to create a time-frequency diagram, see Part 4.2 b). Compare the DFT-CLMS to the adaptive AR-spectrum analyser in Part 4.2. Explain why the spectrum you obtained from the weights of the DFT-CLMS does not resemble to the true power spectrum? [15]

d) Implement the DFT-CLMS for the EEG signal `POz` used in Part 1.4. To reduce computational burden, choose any segment `POz` of length 1200, e.g. `POz(a:a+1200-1)`. Explain your observation about the time-frequency spectrum of the EEG signal. [5]

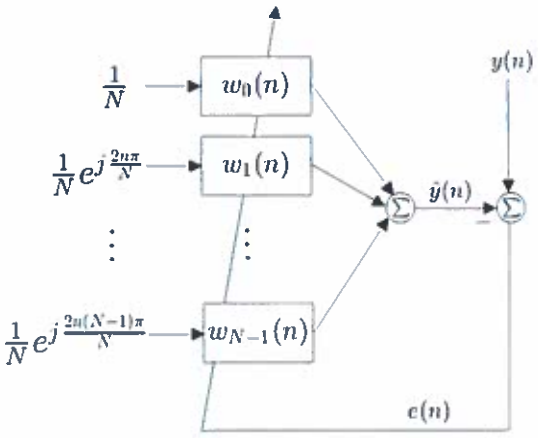


Figure 6: Block diagram of the DFT-CLMS.

References

[1] A. Benveniste, M. Métivier, and P. Priouret, *Adaptive algorithms and stochastic approximations*. Springer Publishing Company, Incorporated, 2012.

[2] W.-P. Ang and B. Farhang-Boroujeny, "A new class of gradient adaptive step-size LMS algorithms," *IEEE Transactions on Signal Processing*, vol. 49, no. 4, pp. 805–810, 2001.

[3] V. Mathews and Z. Xie, "A stochastic gradient adaptive filter with gradient adaptive step size," *IEEE Transactions on Signal Processing*, vol. 41, no. 6, pp. 2075–2087, 1993.

[4] D. Mandic, "A generalized normalized gradient descent algorithm," *IEEE Signal Processing Letters*, vol. 11, no. 2, pp. 115–118, 2004.

[5] D. Mandic, S. Javidi, S. Goh, A. Kuh, and K. Aihara, "Complex-valued prediction of wind profile using augmented complex statistics," *Renewable Energy*, vol. 34, no. 1, pp. 196 – 201, 2009.

[6] B. Widrow, P. Baudrenghien, M. Vetterli, and P. Titchener, "Fundamental relations between the LMS algorithm and the DFT," *IEEE Transactions on Circuits and Systems*, vol. 34, no. 7, pp. 814–820, 1987.